

LiveLane: Formally Verifying Every Edit of an RTL Agent on a Whole RISC-V Core

Jayaraj Jayakumar
UC Santa Cruz
Santa Cruz, CA, USA
jj8@ucsc.edu

Abstract

An agent optimising RTL needs a formal equivalence check, and the standard partition-based checker is least reliable on the strongest agent. LiveLane runs an equivalence check and a sequential check, as composable CHIA nodes, on every iteration of production Gemini agents optimising a whole `ibex_core`. Of 130 optimisation-style rewrites, 49 change what the edited module computes, and synthesis scores 61% of those as better than the original: an objective that rewards removed logic also rewards incorrect logic. Across 9 runs of the strongest model, **all 35 of the 35 edits the checker rejected were correct**, costing 26% of its iterations. An unbounded sequential equivalence check (PDR) of the edited instance from reset, at the parameters the core uses, proves them: in the loop, **81 of 82 rejected edits were proven correct** (the other was a real bug, and it was refuted), and all 69 edits the loop accepted are re-proven from any power-up state. With each package edit checked where its changed types reach, the agent’s re-encodings of shared enums are proven **at the scope of the whole core**, among them the best edit of its run, **6.2% faster and correct, which the checker rejected**. An agent’s tolerance to evaluator latency depends on the model: a slow reasoning model tolerates delay up to a sharp threshold between 120s and 600s, a fast one degrades steadily from under 30s, and latency costs the agent iterations, not per-iteration quality. The check takes 13.9s per iteration, the agent improves the core’s critical path by up to **11.2%** with every accepted edit proven, and two partitioner fixes let eqy complete CVA6’s whole core: 280,764 partitions.

Keywords

agentic EDA, formal equivalence checking, RTL optimisation, CHIA

1 Introduction

An agentic RTL loop proposes a rewrite, evaluates it, and keeps it if the evaluation says it is better. The evaluation is usually simulation and synthesis quality of results (QoR). Both are fast, and both err in the same direction: an edit that removes logic tends to synthesise smaller and faster, whether or not the removed logic mattered, so the objective rewards the incorrect rewrite. Only formal equivalence checking rejects it, and the RTL-editing loops of CHIA [2] do not run one.

LiveLane adds one on every iteration of a production agent optimising a whole RISC-V core, built as CHIA nodes that any RTL-editing loop can adopt. Our results:

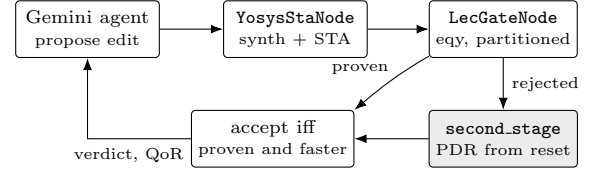


Figure 1: The loop as a CHIA graph. Each box is a node declaring its own resources; the scheduler places them. An edit is accepted only if proven.

- **Synthesis scores reward incorrect edits.** Synthesis scores 61% of the non-equivalent `ibex_core` rewrites as better than the original (§3).
- **The standard checker rejects correct edits.** Of the 35 `gemini-3.1-pro` edits it rejected, all 35 were correct (§6).
- **A sequential check resolves them in the loop.** An unbounded proof from reset proved 81 of 82 rejected edits inside the loop (the other was a real bug, and it was refuted), and re-proves all 69 accepted edits from any power-up state (§7).
- **Proofs beyond instance scope.** Enum re-encodings in the core’s shared package are proven at the scope of the whole core in seconds, including a 6.2%-faster edit the checker rejected (§6).
- **Latency requirement.** The check must be faster than the agent’s own turn, not a human’s two seconds: latency costs an agent iterations, not per-iteration quality (§4).
- **Scale.** Two partitioner fixes take CVA6’s whole core from zero partitions to 280,764 (§5).

For CHIA, this moves formal verification from sign-off into the loop’s critical path: `LecGateNode` and `second_stage` check any RTL edit with open-source tools, and `DelayNode` lets any loop measure the cost of its own evaluator latency.

2 The loop and the equivalence check

One optimisation agent edits RTL. The equivalence check proves or refutes each edit against its parent; a synthesis step scores it against one pinned sky130 Liberty library. An edit is kept only if it is proven equivalent and scores better. The check is built on `eqy`, which splits two designs into partitions and discharges each as an independent SMT obligation; every partition’s miter is flattened before solving, so no proof assumes another partition equivalent. Around it we add an elaboration resolver, a partition-level proof cache keyed on

canonicalised RTLIL and on the query settings (strategy, tool versions), and `DelayNode`, which adds a controlled latency to any CHIA evaluator. The verdict is four-valued; only **proven** accepts an edit, so a timeout is never treated as equivalence.

3 Synthesis scores reward incorrect edits

Instructions are not verification. We ran 5 independent optimisation runs of 2 production models on the whole `ibex_core` (85 iterations). The system prompt states that every edit is checked for equivalence and that an edit changing behaviour is rejected: “*optimise the structure, not the function.*” Even so, `gemin-2.5-flash` proposed **4 edits that change what their module computes** among 27 that reached the check, each refuted by a counterexample from reset (§6) and each described by the model as a restructuring (“refactored”, “inlined”). `gemin-3.1-pro` proposed none among 42 here (2 among 488 in the runs of §7). The error rate depends on the model; only a formal check identifies the errors.

Rejected edits can also score better. `gemin-3.1-pro` proposed, on `picorv32`, to “factor [a] late signal to the top of the logic tree”; simulation passed (1.31 s), synthesis scored it **13.8% faster** (10.31 vs 11.97 ns), and the check rejected it in 1.48 s on `picorv32.mem.done`. To measure how often a non-equivalent edit is rewarded, we generate optimisation-style rewrites of a whole `ibex_core` (9,521 cells, 82,326 μm^2 , 24.50 ns critical path, 9,257 partitions, commit 23806e2; the agent runs use the later pinned release, 18.82 ns and 11,117 partitions) in two families. **Equivalence-preserving** rewrites commute the operands of `&&`, `||`, `&` or `==`; the checker must prove every one. **Simplifying** rewrites do what a QoR-seeking agent does: drop a conjunct or disjunct, remove an inversion, fold a guard to true, swap the two inputs of a multiplexer. Sites are sampled with a fixed seed; every operator matches only a parenthesised expression of two simple operands or a whole ternary right-hand side, so operand boundaries are unambiguous. Sites in comments, in preprocessor branches excluded under the defines in use, in loop bounds, on parameters, and on reset or clock nets are excluded: these are structural edits no optimiser makes, and a rewrite that holds every flip-flop in reset would collapse the core and trivially win on area.

simplifying rewrites	75
rejected by the check	50
proven equivalent (edited logic dead or masked)	25
rejected rewrites that synthesise	50
scored better on QoR (area or delay)	31 (62%)
faster critical path	20 (40%)
smaller area	17 (34%)

62% of the rewrites the check rejects score better than the original (Fig. 2). A loop that accepts on QoR keeps all of them. The largest single timing gain comes from a rejected rewrite: fold guard in `ibex_decoder.sv:647`, if (`illegal_c_insn_i`) $\rightarrow$ if (`1'b1`), which cuts the critical

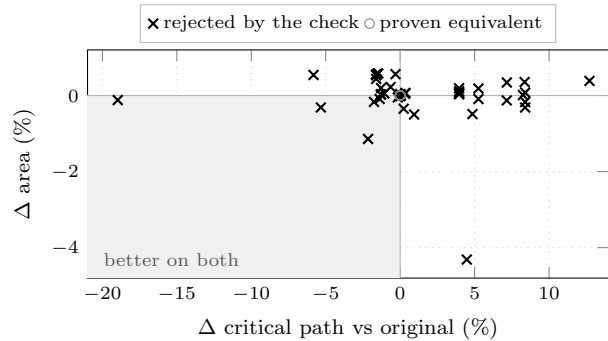


Figure 2: Each simplifying rewrite of `ibex_core` against the original. A QoR-driven loop would keep the rejected rewrites left of or below the axes, including the one with the largest timing gain.

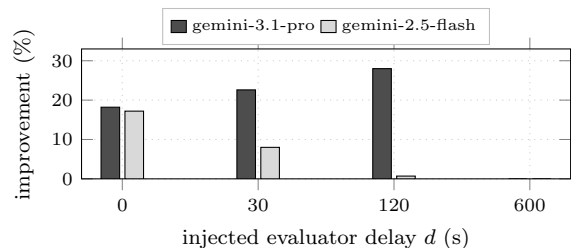


Figure 3: Tolerance depends on the model. Critical-path improvement, fixed 1800 s budget per run (`picorv32`, 2–3 seeds per condition). Turn time: 162 s (pro), 33 s (flash); LiveSim’s human target: 2 s.

path by 19.0% to 19.85 ns. Synthesis is correct here: removing logic does make the circuit faster. The objective is the problem, and only a semantic check fixes it.

4 Latency tolerance of the agent

LiveSim [1] targets two seconds because that is responsive for a human user; an agent does not lose focus while it waits. We treat evaluator latency as a controlled variable: condition $I(d)$ is the identical tool stack plus d seconds of injected delay, and the agent cannot observe its condition (byte-identical prompts, no delay or clock field in its context, both enforced by tests).

For `gemin-3.1-pro`, delays of 0, 30 and 120 s give similar outcomes and 600 s finds no improvement (Fig. 3). Given the same 8 iterations, however, the 600 s condition reaches 18.5%, with no significant difference across the four delays ($p = 0.11$): latency costs an agent iterations, not per-iteration quality. This threshold depends on the model, not on the tool. `gemin-2.5-flash`, whose turn takes 33 s against 162 s for pro, falls from 17.2% to 8.0% at 30 s and to 0.7% at 120 s. Pro tolerates delays up to $0.7\times$ its own turn and finds no improvement at $3.7\times$, a sharp threshold; flash degrades steadily from $0.9\times$ its turn. For both, the bound is the model’s own

turn time: the check need not reach LiveSim’s two seconds, but must stay below the agent’s turn (minutes for a slow reasoning model, tens of seconds for a fast one).

5 Where formal checking spends its time

On a XiangShan Alu iteration the equivalence check takes 54–77% of the wall time and synthesis 1–2% (on `picorv32`, 14% and 2%), so the check is the evaluator worth optimising. Within the check, on a whole `ibex_core` (12 corpus instances, 5,261–15,247 partitions):

phase	cold cache	warm cache
<code>eqy -m</code> (elaborate + partition)	100 s	148 s
prove	228 s	25 s
other	60 s	54 s
wall, 12 instances	388 s	227 s

Proving is 11% of the cached run; `eqy -m` is 65.5%, and partitioning is 92% of that step. **Roughly 60% of the check’s own run time is one single-threaded C++ pass.** A proof cache removes only solver work: given the warm run’s other phases, the bound is $388/(227-25) = 1.93\times$, and we measure $388/227 = 1.71\times$.

Profiling that pass found a debug *Fragment-Partition-Matrix* whose nested loop over partitions $\times$ fragments issues one `log()` per cell, unconditionally. Making it conditional takes `merge_partitions` from 3.65 s to 0.019 s (190 $\times$) and the partition step from 5.36 s to 1.72 s (3.1 $\times$). The same defect explained a result we had recorded as a scaling limit: CVA6’s whole core produced **zero** partitions in 17 min while writing a 2.2 GB log. A second defect also blocked it: generate-block nesting produced partition names longer than the 255-byte filename limit, aborting after 214,903 partitions. With both fixed (names over 100 characters become an FNV-1a digest plus a readable tail; 74,005 shortened, **zero collisions**), the partitioner completes `picorv32` (590 partitions), `ibex_core` (5,261–15,247), XiangShan `DivUnit` (16,065) and **CVA6’s whole core: 280,764 partitions in 790 s**. Both limits were defects in the tool, not properties of the design.

6 Accuracy of the check

The partition-based check rejects correct edits. A partition-based checker cuts the design at signals it matches between the two sides and proves each partition with those cut points as free inputs, so a cut errs towards rejection: an edit that redefines a matched internal net while preserving every output is rejected on that net. Of the agent’s 21 rejected edits, **17 (81%) are correct**, including **all 15 of gemini-3.1-pro’s**. Its rewrites of the fetch FIFO substitute the definition of `out_valid_o` into `pop_fifo`; `eqy` cuts at `out_valid_o`, treats it as a free input, and rejects `fifo_i.pop_fifo`, the only failing partition. All 17 are proven from reset for any power-up state, and 16 also without reset, from any shared initial state.

A sequential check resolves them. On a rejection, we check the edited instance against its parent with an unbounded sequential engine (PDR), falling back to bounded

model checking for a counterexample when PDR times out. The initial state is explicit: both copies start in the same arbitrary state (register pairs matched by name and width) and reset is asserted in the first cycle; a second proof omits the reset. The common shortcut, starting both copies from all-zero, is both unsound and incomplete: it accepts an edit that relies on un-reset flip-flops powering up to zero, and refutes a one-hot state machine, whose all-zero state is illegal. The instance is extracted from the whole core elaborated with hierarchy preserved, so it carries the parameters the core gives it; checked standalone at default parameters, one agent edit to `ibex_counter` is reported proven although it breaks the core’s `minstret` counter. The sequential check resolves all 21 + 50 rejections at a median of 6.3 s each (90th percentile 13.1 s): it refutes 4 agent edits and 49 rewrites with counterexamples from reset, and proves 17 agent edits and 1 rewrite correct. A refutation holds at instance scope: the module computes something different, but the core need not exercise the difference; three LSU rewrites that drop `||pmp_err_q` are correct in this core, whose PMP is disabled. It proves all 48 equivalence-preserving rewrites of instantiated logic from any shared initial state, with **0** counterexamples. Restricted to confirmed bugs, synthesis still scores 30 of 49 (**61%**) better than the original.

Edits to a shared package. Some of the stronger model’s edits change no module: 12 re-encode enums in `ibex_pkg` (`ctrl_fsm_e`, `exc_pc_sel_e`, `imm_b_sel_e`, `pc_sel_e`), all in one run. A package has no instance to extract; `eqy` accepted none, nor could the initial version of the sequential check (all-zero initial state, instance scope). We check such an edit at the scope its changed types reach: one module at its instances, or the top level for a type carried across ports. **12 of 12 are proven correct**, 10 at the scope of the whole core (median 3 s, at most 17 s): with every register matched, we use register correspondence [4], replacing each register pair with one shared free value and proving one cycle, so unchanged logic merges away; the same method fails, as it must, on a moved CSR address. A one-hot controller state machine, whose state register has no counterpart, is proven from reset by PDR. 3 were faster; the best scored 16.93 ns against the 18.04 ns of its parent, **6.2%, the best edit of its run**, which ended at 4.1%; this edit alone would take it to 10.1%.

Real bugs and localisation. On 12 merged lowRISC/ibex bug-fix pull requests, each checked at its own base commit, the check refutes 11 of 12; the twelfth (#1780) fixes PMP logic this configuration never instantiates. Preserving module hierarchy through the front end (`read.slang --keep-hierarchy`) moves partition cuts onto module ports:

	flattened	hierarchical
verdict agreement	12 / 12	
failing partitions in the changed module	47.2%	100%
distinct modules implicated (mean)	2.18	1.00
partitions	9,110	6,022
checked iteration, median	20.6 s	15.2 s

Flattened checking finds the same nets but reports them wherever the partitioner cut (on one instance, in five modules the patch never touched). Hierarchical checking attributes **every one of 55** failing partitions to the module that produced the difference. On the rewrites, all 50 rejections fail only inside the rewritten module (46 matched automatically, the rest by partition name).

7 The complete loop

Across the runs of §3, 5 of 5 improved `ibex_core`'s critical path, by up to **11.2%** (18.82 ns $\rightarrow$ 16.72 ns), through 6 accepted edits, each proven equivalent to its parent before acceptance. The check took **13.9 s** per iteration (warm cache) over 11,117 partitions (86 s at most on a cold first proof), about 13% of a median 126 s iteration and well below the threshold of §4. Total model cost was \$13.01. We then ran the sequential check in the loop: 9 runs of `gemini-3.1-pro` with its initial version (all-zero initial state, instance scope), 9 without it. Without it, correct edits that eqy rejected took 26% of the model's iterations. With it, the agent was told 20 times that a rejected edit was correct (all 20 re-proven from reset), but those rewrites were never faster, and the median improvement barely changes (6.5% vs 6.2%): the check changed what the agent was told, not what it kept. The faster correct edits were the enum re-encodings, provable only at the scope of their types (§6). With the final version of the sequential check (from reset, with package edits checked at the scope their changed types reach) in 18 more runs, 81 of 82 rejected edits came back proven and the agent was told so; the other was a real bug, refuted with a counterexample from reset. It proposed no package edit, and the median is 6.2%. Every edit the loop accepted in these runs is re-proven from reset: 69 of 69.

8 Contribution to CHIA

The track asks for agentic formal verification on a large RTL IP block. LiveLane applies it to a whole RISC-V core on every iteration: 11,117 partitions of `ibex_core` proven per edit in 13.9 s, inside a CHIA graph whose nodes declare their own resources. Three components carry over to any CHIA loop that edits RTL. `LecGateNode` accepts an edit only if it is proven. `second_stage` turns the checker's rejections into proofs or counterexamples from reset, so the agent receives a correct verdict, which matters most for the strongest models. `DelayNode` makes evaluator latency an experimental variable, so a loop can measure the speed it needs. All of it runs on open-source tools (Yosys, eqy, SymbiYosys, OpenSTA), with no commercial licence in the loop.

9 Related work

Scalable equivalence checking merges internal nets proven equal and treats them as cut points [3]; sequential checkers exploit the same structural similarity [4]. The false rejections of §6 are the known cost of that abstraction. We show where it occurs in agentic loops: an LLM that substitutes a signal's definition into a logic cone produces exactly such edits, and

the stronger model produced more. Our sequential check is unbounded model checking [5, 6] per instance, at the design's own parameters. The flow is built on Yosys [7], eqy and SymbiYosys; the cores are Ibex [8] and CVA6 [9]. LiveSim [1] targets two seconds for humans; §4 measures an agent.

10 Scope

The equivalence-checking results are on `ibex` (46 agent runs, 4 Gemini models, plus the rewrite campaign); the latency study is on `picorv32`. Sequential proofs take free inputs at their scope's boundary, so a proof covers every input the rest of the core could supply, and a refutation shows only that the logic within that scope changed.

11 Artifact and reproducibility

The loop is five CHIA nodes, each declaring its own resources and usable alone: `LecGateNode` and `second_stage` (eqy), `YosysStaNode`, `VerilatorSimNode` and `DelayNode`. Any RTL-editing CHIA loop can adopt the two formal nodes unchanged; `DelayNode` lets any loop measure how much evaluator latency its agent tolerates. The artifact contains the loop, the nodes, the rewrite generator, the re-checker for every rejection, the corpus harness, the partitioner fixes as a patch, and every cited measurement file; a script generates the paper's numbers for the rewrite campaign, agent runs, sequential check and latency study from those files. 448 tests pass with the real tools on PATH; CI runs the formal tests, including the whole-core proofs, on every push with the unmodified OSS CAD Suite.

Acknowledgement of AI assistance

Claude, Gemini was used for writing and refactoring code, running and analysing the measurement campaign, and drafting the paper. All experimental design decisions, all measurements, and the final content are by human.

References

- [1] H. Skinner, R. T. Pogniolo, S. Wang, and J. Renau. LiveSim: A Fast Hot Reload Simulator for HDLs. *ISPASS*, 2020.
- [2] CHIA: an open framework for agentic HW/SW co-design flows. <https://chialoops.ai>.
- [3] A. Kuehlmann and F. Krohm. Equivalence checking using cuts and heaps. *DAC*, 1997.
- [4] C. A. J. van Eijk. Sequential equivalence checking based on structural similarities. *IEEE TCAD*, 19(7), 2000.
- [5] A. R. Bradley. SAT-based model checking without unrolling. *VMCAI*, 2011.
- [6] N. Eén, A. Mishchenko, and R. Brayton. Efficient implementation of property directed reachability. *FMCAD*, 2011.
- [7] C. Wolf. Yosys Open SYnthesis Suite. <https://yosyshq.net/yosys/>.
- [8] lowRISC. Ibex RISC-V core. <https://github.com/lowRISC/ibex>.
- [9] F. Zaruba and L. Benini. The cost of application-class processing: energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology. *IEEE TVLSI*, 27(11), 2019.